

CCOS Gap Brief Part 2: People, Housekeeping, Operations and Insights

Oct 8, 2026 · @yvesdlf

1. Purpose and scope

This is Part 2 of the CCOS gap work. Part 1 (the "CCOS Gap Implementation Brief for Claude Code") covered recipes, inventory, procurement, finance and HACCP. Part 2 covers the remaining workspaces: People, Housekeeping and Maintenance, Culinary Operations, Insights, and platform-wide features.

As in Part 1, the gaps come from studying the feature sets of open-source projects on GitHub. No code, schema, prompts or UI text are to be copied from them.

How Claude Code should use this brief

1. Finish, or at least plan, Part 1 first. Part 2 depends on Part 1 tables: locations, audit log, attachments, custom fields, stock movements, approvals and notifications.
2. Read this brief together with `CLAUDE.md`, the Design Bible, `docs/PROGRESS.md`, the ADRs, the specification documents and `docs/IMPLEMENTATION_PLAN.md` if it exists.
3. Reconcile with what already exists. Table names here are proposals; reuse existing tables and conventions.
4. Add Part 2 to `docs/IMPLEMENTATION_PLAN.md` as Phases 6 to 10, following the build order in section 8.

Scope note on housekeeping. CCOS started as a food and beverage platform. Kitchen and front-of-house cleaning is already covered by the HACCP cleaning schedule in Part 1. Hotel room housekeeping in section 4 is an expansion module for hotel and resort customers. Build it as a separately licensable workspace that can be switched off per organization.

2. Reference repos studied

No open-source project covers hospitality operations end to end. The closest is URY (restaurant POS, kitchen display and prep on top of ERPNext). The rest are strong in one area each. Stars and activity were checked on GitHub on 8 October 2026.

Area	Repo	Stars	Licence	Features studied
HR and payroll	frappe/hrms	8.9k	GPL-3.0	Employee lifecycle, shifts, attendance, leave, payroll, appraisals, expense claims
HR	horilla/horilla-hr	1.5k	LGPL-2.1	Onboarding and offboarding checklists, attendance, leave, performance, assets issued to staff
HR	orangehrm/orangehrm	1.1k	GPL-3.0	Employee records, recruitment pipeline, document expiry
Leave	urlaubsverwaltung/urlaubsverwaltung	301	Apache-2.0	Leave balances, sick notes, two-stage approval, holiday calendars, substitutes
Time tracking	kimai/kimai	5.1k	AGPL-3.0	Clock-in/out, timesheets, approval, exports

Area	Repo	Stars	Licence	Features studied
Rostering	TimefoldAI/timefold-solver and quickstarts	1.8k	Apache-2.0	Constraint-based employee scheduling (skills, availability, fairness, labour rules)
Training	frappe/lms	3.3k	AGPL-3.0	Courses, quizzes, progress, certificates
Maintenance	Grashjs/cmms (Atlas CMMS)	805	AGPL-3.0	Work orders, work requests, assets, preventive maintenance schedules, meters, parts, QR codes
Hotel PMS	Qloapps/QloApps	14.4k	OSL-3.0	Room types, room status, reservations
Hotel PMS	Kamra-PMS/kamra-pms	63	AGPL-3.0	Room board, housekeeping status, restaurant POS link, audit trail for human and AI actions, MCP tools

Area	Repo	Stars	Licence	Features studied
Restaurant ops	ury-erp/ury	382	AGPL-3.0	POS, kitchen display, recipe-to-prep link, pre-shift checklists, delay alerts, daily P&L
Analytics	apache/superset	75k	Apache-2.0	Dashboards, scheduled reports, alerts on metrics
Analytics	metabase/metabase	49.6k	AGPL-3.0	Embedded analytics, saved questions, subscriptions
Notifications	novuhq/novu	40k	mixed	Notification workflows, in-app inbox, digests, per-user channel preferences
Forms	formbricks/formbricks	13k	mixed	Form builder, logic, staff and guest surveys

Only Timefold (Apache-2.0) and the leave tool urlaubsverwaltung (Apache-2.0) are licensed so that code could be reused in a closed product, and Timefold is Java, which does not fit the CCOS stack. Everything else is a feature reference only.

3. People workspace

The People workspace should cover the staff lifecycle, rostering, time and attendance, leave, labour cost and training. Payroll calculation itself stays out of scope: CCOS exports approved hours and allowances to a payroll provider, as decided in the earlier competitive analysis. Labour cost next to food cost (prime cost) is the main reason restaurants will want this in CCOS rather than a separate HR tool.

3.1 Staff records

Seen in frappe/hrms, Horilla and OrangeHRM.

Table	Key columns
staff	user_id (nullable: not every staff member logs in), legal_name, preferred_name, employee_code, home_location_id, department, job_title, employment_type (full-time, part-time, casual, daily worker, intern), start_date, end_date, status, hourly_rate or monthly_salary (restricted columns)
staff_documents	staff_id, document_type (contract, ID, work permit, food handler certificate, health certificate), issue_date, expiry_date, attachment_id
staff_skills	staff_id, skill (grill, pastry, barista, sommelier, banquet), level

- Pay and personal ID columns are visible only to roles with a dedicated HR permission. Use column-level protection through a restricted view, not just UI hiding.
- Document expiry alerts at 60, 30 and 7 days. A food handler or health certificate that has expired flags the staff member on the roster.
- Onboarding and offboarding use the checklist engine (section 7.2) with templates such as "new cook" (uniform, locker, HACCP induction, knife skills check) and "leaver" (return keys and uniform, final hours, revoke access).

3.2 Rostering

Seen in frappe/hrms (shifts) and Timefold (optimised scheduling).

Table	Key columns
shift_templates	location_id, name, start_time, end_time, break_minutes, required_skill, headcount
roster_shifts	location_id, staff_id (nullable for open shifts), date, start_at, end_at, break_minutes, role or station, status (draft, published, swapped, cancelled)
staff_availability	staff_id, weekday or date, available_from, available_to, preference

Table	Key columns
shift_swap_requests	shift_id, from_staff_id, to_staff_id, status, approved_by

- Weekly roster grid by location and station, with drag and drop, copy last week, and publish. Publishing notifies staff.
- Live cost bar while building: scheduled hours × rate, labour cost as % of forecast sales (forecast from section 6).
- Rule checks with warnings: overlapping shifts, minimum rest between shifts, maximum hours per week, missing required skill, expired certificate. Make rest and hours limits configurable per country.
- Auto-fill: a TypeScript heuristic in packages/shared that assigns open shifts by skill, availability, contracted hours and fairness. Do not add a Java solver. Record the approach in an ADR.
- Staff see their shifts in the iOS app, can offer shifts for swap, and pick up open shifts. Swaps need manager approval.

3.3 Time and attendance

Seen in Kimai and frappe/hrms.

Table	Key columns
time_entries	staff_id, location_id, roster_shift_id, clock_in_at, clock_out_at, break_minutes, source (kiosk, mobile, manual), clock_in_photo attachment, status (open, submitted, approved, adjusted) — corrections are new rows

- Clock in from a shared tablet kiosk (PIN plus optional photo) or from the iOS app restricted to the venue location.
- Daily variance view: rostered vs actual for each person (late, early leave, unrostered, no-show).
- A manager approves timesheets per pay period. Approved timesheets are locked.

3.4 Leave and absence

Seen in urlaubsverwaltung and frappe/hrms.

Table	Key columns
leave_types	organization_id, name (annual, sick, unpaid, religious holiday, maternity), paid, accrual rule, requires_document
leave_balances	staff_id, leave_type_id, period, entitled, taken, carried_over
leave_requests	staff_id, leave_type_id, start_date, end_date, half_day, reason, attachment_id, status
public_holidays	country, region, date, name

- Approval through the existing approval engine, with optional two-stage approval (head of department, then HR).
- Approved leave blocks rostering for that person and shows on the roster grid.
- Public holidays per country and region. Indonesia and Bali holidays must be supported, including those that change each year.

3.5 Payroll export

- At the end of each pay period, produce a payroll file per location: approved hours split into ordinary, overtime, night and public holiday, plus leave taken, allowances and service charge points if used.
- Export as CSV first. Add payroll provider adapters later through the integration hub.
- Overtime and penalty rates are configurable rules per organization, not hard-coded.

3.6 Training and certifications

Seen in frappe/lms.

- Short training modules made of text, images, video links and a quiz. Typical uses: HACCP induction, allergen awareness, opening and closing procedures, new dish briefings.
- Assign modules by role. Track completion and quiz score per staff member.
- Completing a module can issue an internal certificate with an expiry date, which feeds the staff document alerts in 3.1.
- New recipes can generate a dish briefing module automatically from the recipe card, photo and allergens.

4. Housekeeping and maintenance

Maintenance applies to every CCOS customer, because every kitchen has equipment that breaks. Room housekeeping (4.1 to 4.3) is the optional hotel expansion module described in section 1.

4.1 Rooms and areas (hotel module)

Seen in QloApps and Kamra PMS.

Table	Key columns
rooms	location_id, number, floor, room_type, status (vacant dirty, vacant clean, inspected, occupied, occupied dirty, out of order, out of service), is_active
public_areas	location_id, name (lobby, pool, gym, restrooms), cleaning frequency
room_status_events	room_id, from_status, to_status, changed_by, changed_at, reason — append-only

- CCOS is not a PMS. Occupancy (arrivals, departures, stayovers) comes from the hotel's PMS through the integration hub, or is entered manually. Design an inbound webhook and a manual "today's movements" screen.
- Room board: a colour-coded grid of all rooms by floor showing status, attendant and priority (early arrival, VIP, late checkout).

4.2 Attendant assignment and task board (hotel module)

- Each morning the supervisor builds assignments: rooms per attendant, balanced by credits (checkout room = more credits than stayover). Suggest a balanced split automatically.
- Attendants use the iOS app: list of their rooms, start and finish each room (timed), mark issues, request maintenance, and log minibar consumption and linen changes.
- Finishing a room sets it to "vacant clean". A supervisor inspection with a checklist (section 7.2) sets it to "inspected". A failed inspection sends it back to the attendant.
- Linen and amenity usage per room is recorded as stock consumption, using Part 1 stock movements, so housekeeping supplies are costed and reordered like food.

4.3 Lost and found (hotel and restaurant)

Table	Key columns
lost_found_items	location_id, found_at, found_by, where_found, description, photo attachment, storage_location, status (stored, claimed, returned, disposed, donated), claimed_by_name, released_by, released_at

Retention period is configurable (default 90 days), with a disposal list when it expires. Releasing an item requires the claimant's name and a signature or photo.

4.4 Equipment register

Seen in Atlas CMMS.

Table	Key columns
assets	location_id, parent_asset_id, name, category (cooking, refrigeration, dishwashing, HVAC, extraction, plumbing, electrical, room fittings), manufacturer, model, serial, purchase_date, purchase_cost, warranty_until, supplier_id, manual attachment, qr_code, status
asset_meters	asset_id, meter_type (hours, cycles), reading, read_at

- Each asset gets a printable QR label (reuse Part 1 labels). Scanning it opens the asset page, its history and a "report a problem" button.
- Link refrigeration assets to `haccp_equipment` from Part 1, so a failed temperature log can raise a work order in one tap.

4.5 Work requests and work orders

Seen in Atlas CMMS.

Table	Key columns
work_requests	location_id, asset_id (optional), area, description, photo, priority, requested_by, status (new, accepted, rejected, converted)

Table	Key columns
<code>work_orders</code>	<code>request_id</code> , <code>asset_id</code> , title, description, priority (urgent, high, normal, low), <code>assigned_to</code> (staff or contractor supplier), <code>due_at</code> , status (open, in progress, on hold, waiting for parts, completed, verified), <code>downtime_minutes</code> , <code>labour_minutes</code> , cost
<code>work_order_parts</code>	<code>work_order_id</code> , <code>product_id</code> , quantity — consumes Part 1 stock

- Any staff member can raise a request from the app with a photo. A maintenance lead converts it to a work order.
- Contractors are suppliers in CCOS. A contractor job can create a PO (Part 1) for the repair.
- Spare parts are products in Part 1 inventory, issued through stock movements.
- Completion requires notes and a photo. The requester or a manager verifies it.

4.6 Preventive maintenance

Table	Key columns
<code>pm_schedules</code>	<code>asset_id</code> , task title, checklist template, trigger (every N days, every N meter units), <code>next_due_at</code> , <code>assigned_role</code> or contractor

- A scheduler creates work orders from PM schedules ahead of the due date. Typical examples: hood filter degreasing, combi oven descaling, ice machine sanitising, refrigeration coil cleaning, fire suppression service.
- Reports: overdue PM, mean time between failures and repair cost per asset, and a repair-or-replace flag when cumulative repair cost passes a set share of the asset's purchase cost (default 50%).

5. Culinary operations

These items are already in the CCOS backlog (production planning, menu engineering, nutrition, allergens). URY shows how they connect: menu items map to recipes, recipes drive daily prep, and sales consume stock. CCOS should build that chain on the Part 1 cost engine and stock ledger.

5.1 Menu items and outlets

Seen in URY.

Table	Key columns
<code>menus</code>	<code>location_id</code> , <code>name</code> (all-day, dinner, banquet, room service), <code>active_from</code> , <code>active_to</code>
<code>menu_items</code>	<code>menu_id</code> , <code>recipe_id</code> , <code>display_name</code> , <code>category</code> , <code>selling_price</code> , <code>tax_rate_id</code> , <code>pos_item_code</code> , <code>is_available</code>
<code>menu_item_modifiers</code>	<code>menu_item_id</code> , <code>name</code> , <code>recipe_id</code> or <code>product_id</code> , <code>quantity</code> , <code>price_delta</code>

- One recipe can appear on several menus at different prices and portions per outlet.
- `pos_item_code` maps menu items to POS sales lines (section 6.2).
- Modifiers (extra cheese, no onion, add avocado) adjust both price and theoretical consumption.

5.2 Production planning and prep lists

Seen in URY ("integrated with production planning for daily prep").

Table	Key columns
<code>prep_plans</code>	<code>location_id</code> , <code>date</code> , <code>status</code> (draft, issued, completed)
<code>prep_plan_lines</code>	<code>prep_plan_id</code> , <code>recipe_id</code> (sub-recipe), <code>station</code> , <code>par_qty</code> , <code>on_hand_qty</code> , <code>required_qty</code> , <code>assigned_to</code> , <code>done_qty</code> , <code>done_at</code>

- Required quantity = forecast need (from the sales forecast in 6.3 and banquet events) + par level – on hand, expanded through the recipe dependency graph so sub-recipes are planned before the dishes that use them.
- Prep lists by station on the kitchen tablet, with tick-off and actual yield entry.
- Completing a prep line posts `production_consume` and `production_output` stock movements (Part 1) and can print prep labels.
- Yield variance (actual vs recipe yield) is recorded per batch and flagged when it drifts beyond a threshold, because it signals portioning or trimming problems.

5.3 Menu engineering

- Classic menu engineering matrix per menu and period: popularity (share of items sold) against contribution margin (price minus theoretical cost). Quadrants: stars, plowhorses, puzzles, dogs.
- Inputs come from POS sales (6.2) and the cost engine. Recipe cost changes update the matrix automatically.
- Suggested actions per quadrant (reprice, re-portion, reposition on the menu, remove), shown as suggestions only.

5.4 Allergens and nutrition

The source workbooks already hold allergen and nutrition data, so this mainly needs building on the existing dependency graph.

- Products carry allergens (union of the EU 14 and US 9 lists, configurable per organization) and nutrition per 100 g.
- Recipes inherit allergens and nutrition through sub-recipes automatically, adjusted for yield. A "may contain" flag is kept separate from "contains".
- Allergen matrix per menu (dishes × allergens) as a printable PDF for front of house, and allergen fields on prep labels.
- Any recipe or supplier product change that alters a dish's allergens notifies the head chef and flags the menu matrix as needing re-issue.

5.5 Kitchen display (later)

Seen in URY Mosaic and OpenKDS: order tickets by station with live status, bump, recall and timing alerts.

A kitchen display only makes sense with a live POS order feed. Design the event interface (order created, item modified, item cancelled) now, but build the display itself after at least one POS integration exists. Until then, prep lists cover the kitchen tablet.

6. Insights workspace

Insights turns the data from every other workspace into the few numbers an operator checks daily. Superset and Metabase show the patterns: saved metrics, dashboards, scheduled email reports and alerts when a metric crosses a threshold. Build these natively rather than embedding a BI tool, so they follow CCOS permissions and the Design Bible.

6.1 Metrics layer

- Define each KPI once, as a SQL view or Postgres function in a dedicated `insights` schema, respecting RLS. The UI and reports read only from this layer.
- Document every metric's formula in `docs/METRICS.md` so numbers are consistent across screens.

6.2 Sales data

Table	Key columns
<code>sales_lines</code>	<code>location_id</code> , <code>business_date</code> , <code>pos_item_code</code> , <code>menu_item_id</code> , <code>quantity</code> , <code>gross_amount</code> , <code>discount</code> , <code>net_amount</code> , <code>service_charge</code> , <code>tax</code> , <code>covers</code> , <code>source</code> (POS adapter, CSV import)

- Start with a CSV import of the POS daily item sales report, mapped to menu items by `pos_item_code`. Add POS adapters later through the integration hub.
- Importing sales posts `sale_consume` stock movements from recipes (theoretical consumption), including modifiers.

6.3 Core KPIs

KPI	Formula
Actual food cost %	$(\text{opening stock} + \text{purchases} - \text{closing stock}) \div \text{food sales}$
Theoretical food cost %	$\Sigma (\text{items sold} \times \text{recipe cost}) \div \text{food sales}$
Food cost variance	actual – theoretical, in % and money, broken down by product
Labour cost %	$\text{approved labour cost (section 3)} \div \text{net sales}$
Prime cost %	$(\text{food and beverage cost} + \text{labour cost}) \div \text{net sales}$
Sales per labour hour	$\text{net sales} \div \text{approved hours}$
Waste %	$\text{waste value} \div \text{food sales}$, by reason
Average spend per cover	$\text{net sales} \div \text{covers}$
HACCP compliance %	from Part 1
Maintenance backlog	open work orders and overdue PM

KPI	Formula
Rooms cleaned per attendant hour	hotel module only

Theoretical vs actual variance by product is the most valuable report in this list. It shows where stock is going missing, being over-portioned or wasted.

6.4 Sales forecast

- Forecast covers and item sales per day and service period from the same weekday over the last 4 to 8 weeks, adjusted for public holidays and known events (banquets, bookings entered manually).
- Keep the method simple and explainable. It feeds rostering targets (3.2), prep plans (5.2) and order suggestions (Part 1 auto-reorder).
- Show forecast vs actual so managers can judge its accuracy.

6.5 Dashboards, scheduled reports and alerts

- Role-based home dashboards: owner (group roll-up across outlets), general manager (outlet), head chef (food cost, waste, prep, HACCP), HR (labour, leave, expiring documents).
- A daily flash report emailed each morning: yesterday's sales, covers, labour %, food cost estimate, waste, HACCP failures and open issues.
- Metric alerts with thresholds per organization, e.g. labour % above target, theoretical vs actual variance above 2 points, waste above target.
- Every table and chart can be exported to CSV and PDF.

7. Platform-wide features

Several modules above need the same building blocks. Build these once, before the modules that use them.

7.1 Notifications

Seen in Novu.

Table	Key columns
notifications	organization_id, recipient_user_id, event_type, title, body, entity_type, entity_id, severity, read_at, created_at

Table	Key columns
<code>notification_preferences</code>	<code>user_id</code> , <code>event_type</code> , channels (in-app, push, email), <code>quiet_hours</code>

- One `notify(event_type, recipients, payload)` function used by every module. Recipients can be users, roles or "duty manager at location".
- In-app inbox with unread count, iOS push through Capacitor, and email.
- Daily digest option for non-urgent events. Critical events (HACCP critical failure, equipment down) ignore quiet hours.

7.2 Checklists and forms builder

Seen in Formbricks, Horilla (onboarding) and URY (pre-billing checklists).

Table	Key columns
<code>checklist_templates</code>	<code>organization_id</code> , name, category, items (jsonb: question, type, required, pass criteria, photo required), version
<code>checklist_runs</code>	<code>template_id</code> , version, <code>location_id</code> , <code>entity_type</code> , <code>entity_id</code> , <code>started_by</code> , <code>completed_at</code> , score, status
<code>checklist_answers</code>	<code>run_id</code> , <code>item_key</code> , value, passed, note, <code>attachment_id</code>

- Item types: yes/no, number with range, text, choice, photo, signature, temperature (writes a HACCP log).
- Uses: opening and closing checklists, room inspections, onboarding, PM tasks, line checks before service, guest and staff surveys.
- Templates are versioned so past runs keep their original questions. Failed items can create a task or work order automatically.

7.3 Tasks

- A general `tasks` table (title, assignee or role, location, `due_at`, entity link, status) used for one-off jobs and for anything generated by checklists, alerts or schedules.
- "My day" screen in the iOS app combining tasks, HACCP checks, prep lines, cleaning and work orders for the signed-in person.

7.4 SOP library

- Rich-text SOPs with photos and video links, organised by department, linked to recipes, equipment and checklists.
- Versioned, with an approval step before a new version is published, and a "read and acknowledged" record per staff member for critical SOPs.
- Available offline on the iOS app.

7.5 Team communication

- Pre-shift briefing notes per location and date (specials, 86'd items, VIPs, events), shown on the "My day" screen.
- Comments with @mentions on any entity (recipe, work order, PO, incident). Do not build a general chat tool.

7.6 Incidents

- Incident log for guest complaints, staff injuries, near misses, allergen incidents and security events: what, where, who, severity, photos, actions taken, follow-up owner.
- Restricted visibility by incident type, with an investigation and close-out step through the approval engine.

7.7 Integrations and AI

- All inbound and outbound connections (POS sales, PMS occupancy, payroll export, accounting journals) go through the integration hub with the webhooks from Part 1, each adapter isolated behind one interface.
- Expose CCOS actions to AI agents through an MCP server with the same RBAC as the UI, as Kamra PMS does. Every AI action is recorded in the audit log with the human it acted for. Money and stock postings always need human confirmation.